Self-Correcting Multi-Agent AI System

A Multi-Agent Self-Correcting Framework for Reliable AI Response Generation Using Claim Verification and Iterative Refinement

---

1. Abstract

Large Language Models (LLMs) have significantly transformed how users interact with artificial intelligence systems by enabling natural language understanding and generation at unprecedented levels. Despite their capabilities, modern AI systems frequently produce outputs that contain hallucinations, unsupported claims, or logically inconsistent statements. These issues limit their adoption in reliability-critical domains such as engineering, research, healthcare, and decision-making environments.

This project proposes a **Self-Correcting Multi-Agent AI System**, designed to improve the reliability of AI-generated responses through structured collaboration among specialized agents. Instead of relying on a single AI model to generate answers, the proposed system decomposes the response generation process into multiple stages, including planning, generation, claim extraction, verification, critique, and refinement.

The system introduces a **claim-level verification pipeline**, where generated content is broken into atomic factual claims that are validated against external evidence sources using retrieval-based verification and semantic entailment techniques. A critic agent evaluates weaknesses in the generated content and triggers iterative refinement until quality thresholds are satisfied.

The final output presented to users contains verified information, confidence scores, and supporting evidence, making the system transparent, explainable, and auditable. The proposed architecture bridges the gap between generative AI systems and trustworthy AI deployment, making it suitable as a research-oriented and industry-relevant final-year project.

---

2. Motivation

Artificial intelligence systems powered by large language models have become widely adopted for tasks such as answering technical questions, generating reports, summarizing research

papers, and assisting in software development. However, these systems are probabilistic generators rather than knowledge-grounded reasoning engines. As a result, they may produce fluent but incorrect responses.

The major challenges observed in current AI systems include:

- Generation of incorrect facts with high confidence.
- Lack of explainability regarding information sources.
- Absence of mechanisms to validate claims before presenting answers.
- Inability to self-correct without human intervention.
- Difficulty in identifying partially correct responses.

In real-world applications, users require answers that are not only coherent but also verifiable and trustworthy. Current AI pipelines typically follow a simple architecture:

User Query → LLM → Generated Response

This approach lacks verification and feedback loops, making it unsuitable for high-reliability environments.

The motivation behind this project is to move from **generation-centric AI systems** toward **verification-centric AI systems**, where correctness and trustworthiness are first-class objectives. By introducing multiple specialized agents responsible for different tasks, the system emulates collaborative reasoning similar to human workflows, where drafts are reviewed, validated, and improved before final delivery.

---

3. Problem Statement

Modern AI-based conversational systems generate responses without validating factual correctness during the generation process. This leads to several problems:

1. **Hallucinated Information** – The AI produces statements that appear correct but lack factual support.
2. **Lack of Evidence Traceability** – Users cannot determine the origin of information.
3. **No Self-Correction Mechanism** – Errors remain unless manually corrected.
4. **Overconfidence in Outputs** – Systems present uncertain information as definitive facts.
5. **Limited Transparency** – Internal reasoning processes are not visible to users.

The problem addressed in this project is:

How can an AI system automatically evaluate, verify, and refine its own responses before presenting them to users, thereby improving reliability and trust without requiring human intervention?

---

4. Objectives

The primary objective of this project is to design and implement a software system capable of producing reliable AI-generated responses through multi-agent collaboration and automated verification.

Primary Objectives

- Design a multi-agent AI architecture for response generation.
- Implement claim extraction from generated text.
- Develop an evidence retrieval and verification mechanism.
- Introduce a critic-refinement feedback loop.
- Provide explainable outputs with verification indicators.

Secondary Objectives

- Reduce hallucinated information in AI responses.
- Improve answer reliability through iterative refinement.
- Provide an auditable workflow for AI decisions.
- Demonstrate measurable improvements over baseline LLM outputs.

Technical Objectives

- Implement asynchronous backend orchestration.
- Integrate retrieval-based verification techniques.
- Develop a modular architecture allowing independent agent execution.
- Support scalability and extensibility for future research.

---

5. Scope of the Project

The scope of this project focuses on building a **software-based AI reliability framework** rather than training a new language model from scratch.

Included in Scope

- Multi-agent orchestration framework.
- Claim-level verification pipeline.
- Retrieval-based evidence matching.
- Semantic verification using entailment models.
- Iterative refinement workflow.
- Web-based dashboard for visualization.

Excluded from Scope

- Training large-scale language models.
- Large-scale distributed deployment.
- Domain-specific fine-tuned LLM training.
- Real-time web-scale indexing.

The system is designed as a **research prototype** that demonstrates architectural feasibility and measurable improvements in reliability.

---

6. Existing System Analysis

Most existing AI systems operate using a single-model architecture where a user query is processed directly by a language model. Even modern Retrieval-Augmented Generation (RAG) systems primarily focus on providing context before generation rather than verifying outputs after generation.

Limitations of Existing Systems

1. **Single-pass generation**
    - Output is generated once without evaluation.
2. **Weak verification**
    - Retrieved documents may be unrelated or misinterpreted.
3. **No claim granularity**
    - Entire answers are evaluated instead of individual facts.
4. **Lack of feedback loop**
    - Systems do not iteratively improve outputs.
5. **Limited explainability**
    - Users cannot see which parts are reliable.

Although recent research introduces multi-agent frameworks, many implementations remain experimental and lack structured verification pipelines.

---

7. Research Gap

Current research and industry systems largely focus on improving generation quality rather than improving verification reliability. The following gaps exist:

- Absence of integrated pipelines combining generation, verification, and refinement.
- Limited adoption of claim-level verification strategies.
- Lack of systems that explicitly distinguish between verified and uncertain information.
- Insufficient focus on explainability in AI-generated outputs.

This project addresses these gaps by introducing a **structured multi-agent pipeline**, where each agent has a clearly defined responsibility and contributes to improving output reliability.

---

8. Proposed System Overview

The proposed system introduces a layered architecture where AI generation is only one component of a larger pipeline.

Instead of directly delivering generated responses, the system performs the following stages:

1. Query analysis and planning.
2. Initial response generation.
3. Claim extraction from generated text.
4. Evidence retrieval for each claim.
5. Claim verification using semantic analysis.
6. Critique of weaknesses and inconsistencies.
7. Response refinement using verified evidence.
8. Final delivery with verification indicators.

This transforms the AI workflow into:

User → Planner → Generator → Claim Extractor
→ Verifier → Critic → Refiner → Final Answer

The system ensures that unreliable content is detected and improved before reaching the user.

---

9. User Personas

9.1 Researcher / Engineer

This user requires technically accurate and verifiable explanations. The system helps by providing evidence-backed responses and highlighting uncertain claims.

9.2 AI Product Developer

This user uses the system as a validation layer before deploying AI-generated content to users. The dashboard allows inspection of verification results and confidence scores.

9.3 Student / Knowledge Seeker

This user benefits from transparent answers where verified and uncertain information is clearly distinguished, improving learning reliability.

Design decisions prioritize clarity, transparency, and explainability across all user types.

---

10. User Flow (Detailed)

The user interaction begins with a query submitted through the web interface or API. The system processes the request through multiple stages while continuously updating system state.

Step-by-Step User Flow

1. The user submits a query through the interface.
2. The system acknowledges the request and initializes a processing pipeline.
3. The planner agent determines how the query should be answered.
4. The generator produces an initial draft response.
5. The claim extraction module identifies factual statements.
6. The verification module searches for supporting evidence.
7. The critic evaluates weaknesses in the response.
8. The refinement agent produces an improved final response.
9. The system displays the final answer with verification labels and evidence references.

The user can optionally inspect intermediate stages through the dashboard, allowing transparency into how the answer evolved.

This section includes:

✅ System Architecture (Layered Design)
✅ Backend Pipeline Design Flow
✅ Agent Communication Architecture

✅ Data Flow Diagrams (Conceptual)
✅ Backend Execution Lifecycle
✅ Database Design & Schemas
✅ Async Queue & Task Management
✅ Internal Module Responsibilities
✅ Detailed Backend Processing Flow

PART 2 — SYSTEM ARCHITECTURE & BACKEND PIPELINE DESIGN

---

11. System Architecture Design

The proposed system follows a **layered architecture**, separating responsibilities into independent modules to ensure scalability, maintainability, and extensibility. Each layer performs a specific role in transforming user queries into verified responses.

The architecture is divided into the following layers:

11.1 Presentation Layer

This layer represents the interaction interface between the user and the system. It includes a web-based dashboard where users submit queries and view results. The interface displays not only the final answer but also verification indicators, confidence scores, and supporting evidence.

Responsibilities include:

- Accepting user queries.
- Displaying processing progress.
- Rendering verified and uncertain claims visually.
- Providing audit logs for transparency.

The frontend communicates with the backend through REST APIs and WebSocket connections for real-time updates.

11.2 API and Orchestration Layer

The API layer acts as the entry point of the system. It receives user requests and initializes the processing workflow. The orchestration component manages communication between agents and ensures that tasks are executed in the correct order.

Responsibilities include:

- Creating workflow instances.
- Maintaining task states.
- Scheduling agents.
- Handling failures and retries.
- Aggregating outputs from different agents.

This layer ensures that the system behaves as a coordinated pipeline rather than independent AI calls.

11.3 Agent Execution Layer

This layer contains specialized agents responsible for individual tasks in the pipeline. Each agent operates independently and communicates through structured inputs and outputs.

Agents included:

- Planner Agent
- Generator Agent
- Claim Extraction Agent
- Retrieval Agent
- Verification Agent
- Critic Agent
- Refinement Agent

The separation of agents enables modular upgrades without affecting the entire system.

11.4 Retrieval and Knowledge Layer

This layer handles information retrieval from internal and external knowledge sources. It combines keyword-based search and semantic search to identify relevant evidence for claims.

Components include:

- Vector database for semantic similarity search.
- Full-text index for keyword matching.
- Evidence ranking module.

- Source credibility evaluator.

11.5 Data Persistence Layer

This layer manages storage of all system artifacts, including queries, intermediate outputs, claims, verification results, and logs.

Data stored includes:

- Task metadata
- Generated responses
- Extracted claims
- Evidence records
- Verification scores
- Agent logs

A relational database is used for structured data, while vector storage handles embeddings.

---

12. High-Level Backend Pipeline Flow

The backend pipeline converts a user query into a verified response through sequential stages. Each stage produces structured outputs that become inputs for the next stage.

Pipeline Overview

User Query

↓

Planner Agent

↓

Generator Agent

↓

Claim Extraction Agent

↓

Evidence Retrieval

↓

Verification Agent

↓

Critic Agent

↓

Refinement Agent

↓

Final Response

The pipeline is designed to allow asynchronous execution so that long-running tasks such as retrieval and verification do not block system responsiveness.

---

13. Agent Communication Architecture

The system uses a **message-driven communication model** rather than direct synchronous calls. Each agent publishes results to a shared task state, which triggers execution of the next agent.

Communication Principles

1. Agents do not directly call each other.
2. The orchestration engine controls execution order.
3. Each agent reads input from the database and writes output back.
4. State transitions determine next execution step.

This approach ensures:

- Fault tolerance
- Easy debugging
- Reproducibility of results
- Independent scaling of agents

Example Workflow State Transitions

CREATED → PLANNED → GENERATED → CLAIMS_EXTRACTED

→ VERIFIED → CRITIC_REVIEWED → REFINED → COMPLETED

---

14. Data Flow Design

The data flow in the system represents how information moves between components.

14.1 Data Flow Level 0 (Conceptual)

At the highest level, the system can be represented as:

User → AI Processing System → Verified Response

Internally, the system expands into multiple stages responsible for analysis, validation, and refinement.

14.2 Data Flow Level 1 (Detailed)

User Input

↓

Query Processor

↓

Generation Module

↓

Claim Analyzer

↓

Evidence Retriever

↓

Verification Engine

↓

Critique Module

↓

Refinement Module

↓

Output Renderer

Each module transforms the input into a more structured and validated form.

---

15. Backend Execution Lifecycle

When a user submits a query, the backend performs the following lifecycle steps:

Step 1 — Workflow Initialization

A new workflow instance is created with a unique identifier. The query and metadata are stored in the database.

Step 2 — Planning Phase

The planner agent analyzes the query and determines execution strategy. Subtasks are generated and queued.

Step 3 — Generation Phase

The generator produces initial responses for each subtask using an LLM. Outputs are stored as draft responses.

Step 4 — Claim Analysis Phase

The claim extraction agent parses generated text into atomic claims. Each claim is stored independently for verification.

Step 5 — Retrieval Phase

Evidence retrieval identifies candidate supporting documents or snippets for each claim.

Step 6 — Verification Phase

The verification agent evaluates whether evidence supports or contradicts claims and assigns confidence scores.

Step 7 — Critique Phase

The critic agent analyzes verification results and identifies weaknesses or unsupported claims.

Step 8 — Refinement Phase

The refinement agent produces a revised response using verified information.

Step 9 — Completion Phase

The final output is returned to the user along with verification metadata.

---

16. Database Design

The system requires structured storage to maintain traceability and reproducibility of AI decisions.

16.1 Core Tables

Workflows Table

Stores user queries and workflow status.

Fields:

- workflow_id
- user_query
- status
- created_at
- completed_at

Responses Table

Stores generated outputs from the generator and refinement agents.

Fields:

- response_id
- workflow_id
- agent_type
- response_text
- model_used
- timestamp

Claims Table

Stores extracted claims.

Fields:

- claim_id
- response_id
- claim_text
- entities
- extraction_confidence

Evidence Table

Stores retrieved evidence.

Fields:

- evidence_id
- claim_id
- source_url
- snippet
- retrieval_score

Verification Table

Stores verification results.

Fields:

- verification_id
- claim_id
- status
- confidence_score

- evidence_id

---

17. Asynchronous Task Processing

The system uses asynchronous processing to handle computationally expensive tasks.

Why Async Processing is Required

- LLM calls are slow and variable.
- Retrieval operations may involve network latency.
- Multiple claims can be verified in parallel.

Queue-Based Execution

Tasks are pushed into a queue where workers process them independently.

Example:

- Generator worker
- Retrieval worker
- Verification worker

This enables parallel processing of multiple claims, significantly reducing total latency.

---

18. Internal Module Responsibilities

Each backend module has clearly defined responsibilities:

Planner Module

Defines execution strategy and constraints.

Generator Module

Produces initial textual output.

Claim Analyzer

Extracts and normalizes claims.

Retriever Module

Fetches candidate evidence.

Verification Module

Evaluates claim validity.

Critic Module

Detects weaknesses and inconsistencies.

Refinement Module

Produces final verified response.

Clear separation ensures modular development and easier testing.

---

19. Backend Processing Flow (Detailed Logical View)

The logical flow inside the backend can be summarized as follows:

1. API receives query.
2. Workflow created.
3. Planner generates subtasks.
4. Generator produces initial answer.
5. Claim extractor identifies facts.
6. Retrieval runs in parallel for all claims.
7. Verification evaluates claims.
8. Critic generates improvement instructions.
9. Refinement agent produces final output.
10. Result stored and returned.

The system maintains logs at every step to allow debugging and explainability.

This part includes:

✅ Claim Extraction Algorithms (technical design)
✅ Verification & Confidence Scoring Model
✅ Model Selection Justification
✅ Dataset Requirements & Preparation

✅ Evaluation Methodology
✅ Experimental Design
✅ Result Analysis Structure
✅ Deployment Architecture
✅ Limitations & Future Scope
✅ Conclusion

PART 3 — ALGORITHMIC DESIGN, EVALUATION & DEPLOYMENT

---

20. Claim Extraction Methodology

The purpose of claim extraction is to convert unstructured generated text into structured, verifiable units of information. Instead of verifying entire paragraphs, the system operates at the level of atomic factual statements.

20.1 Motivation for Claim-Level Processing

Verifying entire answers introduces ambiguity because a single paragraph may contain both correct and incorrect information. By decomposing text into independent claims, the system can:

- Verify information more precisely.
- Identify partially incorrect answers.
- Provide explainability at sentence level.
- Improve refinement accuracy.

20.2 Claim Extraction Pipeline

The claim extraction process consists of multiple stages:

Step 1 — Sentence Segmentation

The generated response is divided into sentences using syntactic parsing rules. Each sentence becomes a candidate for claim extraction.

Step 2 — Entity Identification

Named Entity Recognition (NER) is applied to identify entities such as technologies, organizations, dates, and numerical values. Entity presence increases the likelihood of factual claims.

Step 3 — Claim Identification

An LLM-based extractor identifies statements that assert facts, definitions, or causal relationships. Non-factual statements such as opinions are filtered out.

Step 4 — Claim Normalization

Claims are rewritten into canonical form to remove ambiguity. For example:

Original:

> "TCP usually assumes packet loss means congestion."

Normalized:

> "TCP interprets packet loss as an indicator of congestion."

20.3 Claim Types

The system categorizes claims into:

- **Factual Claims** – Statements describing facts.
- **Numeric Claims** – Quantitative information.
- **Causal Claims** – Cause-effect relationships.
- **Definition Claims** – Explanatory statements.

Different claim types may use different verification strategies.

---

21. Evidence Retrieval Strategy

The retrieval module is responsible for finding candidate evidence that may support or contradict extracted claims.

21.1 Hybrid Retrieval Approach

The system combines two retrieval strategies:

Keyword-Based Retrieval (BM25)

Efficient for exact term matching and technical phrases.

Semantic Retrieval (Embeddings)

Captures contextual similarity even when wording differs.

The hybrid approach ensures high recall while maintaining precision.

21.2 Retrieval Pipeline

1. Claim text converted into embedding vector.
2. Vector search performed in embedding database.
3. Keyword search performed in indexed corpus.
4. Results merged and ranked.
5. Top-K evidence snippets selected.

21.3 Evidence Ranking Factors

Evidence ranking considers:

- Semantic similarity score
- Keyword match score
- Source credibility
- Contextual relevance
- Recency (if applicable)

---

22. Verification and Confidence Scoring

The verification module determines whether evidence supports a claim.

22.1 Verification Objective

Given a claim **C** and evidence **E**, determine:

- Supported
- Contradicted
- Uncertain
- No Evidence

22.2 Natural Language Inference (NLI)

Verification uses entailment models where:

- Evidence acts as premise.
- Claim acts as hypothesis.

The model outputs probabilities for:

- Entailment
- Neutral
- Contradiction

22.3 Verification Score Formulation

A combined score is calculated:

Verification Score =

w1 × Entailment Probability +

w2 × Semantic Similarity +

w3 × Source Credibility

Where:

- w1, w2, w3 are empirically tuned weights.
- Entailment measures logical support.
- Similarity measures contextual alignment.
- Credibility measures reliability of source.

22.4 Decision Thresholds

- Score ≥ 0.75 → Verified
- 0.40 ≤ Score < 0.75 → Uncertain
- Contradiction probability high → Contradicted
- No evidence retrieved → No Evidence

---

23. Critic and Refinement Strategy

After verification, the critic agent evaluates the response holistically.

23.1 Critic Responsibilities

- Detect unsupported claims.
- Identify missing explanations.
- Flag logical inconsistencies.
- Suggest improvements.

The critic generates structured feedback rather than free text.

23.2 Refinement Process

The refinement agent regenerates the response using:

- Verified claims
- Selected evidence snippets
- Critic feedback

This produces a response that is more reliable than the original output.

23.3 Iteration Control

To prevent infinite loops:

- Maximum iterations limited (e.g., 3).
- Refinement stops when verification improvement is minimal.

---

24. Dataset Requirements and Preparation

24.1 Dataset Requirement

Datasets are required primarily for:

- Evaluation of verification accuracy.
- Testing retrieval effectiveness.
- Benchmark comparisons.

The system itself does not require training large models from scratch.

24.2 Recommended Datasets

- **FEVER Dataset** – Fact verification benchmark.
- **Wikipedia Corpus** – General factual evidence.
- **Natural Questions Dataset** – QA evaluation.
- **MultiNLI / SNLI** – Entailment model evaluation.

24.3 Synthetic Dataset Creation

Since real-world queries vary, a synthetic dataset is created by:

1. Generating answers using an LLM.
2. Extracting claims.
3. Manually labeling claims as supported or unsupported.
4. Using labels for evaluation.

---

25. Evaluation Methodology

Evaluation compares two systems:

1. Baseline LLM (no verification).
2. Proposed self-correcting system.

Evaluation Metrics

- Claim Verification Accuracy
- Hallucination Detection Rate
- Precision and Recall
- F1 Score
- Response Helpfulness (Human Rating)
- Latency Overhead

25.1 Hallucination Reduction Metric

Hallucination Reduction =

(Baseline Incorrect Claims − System Incorrect Claims)

/ Baseline Incorrect Claims

---

26. Experimental Design

Experiments are conducted in three phases:

Phase 1 — Retrieval Evaluation

Measure whether correct evidence appears in top-K results.

Phase 2 — Verification Evaluation

Measure accuracy of claim classification.

Phase 3 — End-to-End Evaluation

Compare final answer quality against baseline LLM responses.

Human evaluators rate responses on correctness and clarity.

---

27. Deployment Architecture

The system is deployed as modular services.

Deployment Components

- FastAPI backend container
- Worker containers for agents
- Redis queue
- PostgreSQL database
- Vector database
- React frontend

Deployment Flow

User → Frontend → API Gateway → Agent Workers

↓

Database + Vector DB

Docker-based deployment ensures reproducibility.

---

28. Limitations

Despite improvements, limitations include:

- Dependence on retrieval quality.
- Verification limited by available evidence.
- Increased latency compared to single LLM systems.
- Difficulty handling subjective or opinion-based queries.

These limitations are acknowledged as future research directions.

---

29. Future Scope

Potential extensions include:

- Knowledge graph integration.
- Domain-specific verification models.
- Real-time browser extension for fact checking.
- Learning-based refinement strategies.
- Reinforcement learning for agent coordination.

---

30. Conclusion

This project presents a novel approach to improving reliability in AI-generated responses through a structured multi-agent architecture. By introducing claim extraction, evidence-based

verification, and iterative refinement, the system transforms AI response generation into a verifiable and transparent process.

The proposed framework demonstrates how generative AI systems can evolve from probabilistic text generators into trustworthy information systems. The architecture is modular, extensible, and suitable for future research and industrial applications, making it a strong candidate for a final-year engineering project.

✅ SCENARIO

User asks:

```
Explain why TCP performs poorly in wireless networks and compare it
with QUIC
```

✅ STEP 1 — USER INPUT & WORKFLOW INITIALIZATION

1.1 API Request

Frontend sends request:

```
POST /query
```

Payload:

```
{

  "query": "Explain why TCP performs poorly in wireless networks and
compare it with QUIC"

}
```

1.2 Backend Processing

FastAPI backend performs:

```
Generate workflow_id

Initialize workflow state

Store query
```

Database entry:

workflow_id	query	status
wf_101	Explain why TCP...	CREATED

1.3 System Action

Planner agent job queued:

```
queue.push(planner_agent, wf_101)
```

✅ STEP 2 — PLANNING PHASE

Planner agent starts execution.

2.1 Query Preprocessing (Backend)

Operations:

- normalize text
- detect keywords ("explain", "compare")
- estimate complexity

Intermediate result:

```
intent_hint = explain + compare
```

2.2 Intent Classification (LLM Reasoning)

LLM receives:

```
Classify intent of query.
```

LLM output:

```
{
  "intent": ["explain","compare"],
  "domain": "networking"
}
```

Backend stores intent.

2.3 Task Decomposition (LLM Reasoning)

LLM receives:

```
Break query into independent tasks.
```

Output:

```
1. Explain TCP congestion control

2. Explain wireless packet loss characteristics

3. Explain QUIC protocol

4. Compare TCP and QUIC
```

2.4 Backend Plan Construction

Backend:

- validates tasks
- assigns priority
- enables verification

Plan stored:

```
plan_501
```

Workflow state:

```
CREATED → PLANNED
```

2.5 Next Step Triggered

Generator agent queued.

✅ STEP 3 — GENERATION PHASE

Generator agent executes.

3.1 Prompt Construction (Backend)

For task 1:

```
SYSTEM:

Provide factual technical explanation.


TASK:

Explain TCP congestion control.


CONSTRAINTS:

Avoid speculation. Suitable for verification.
```

3.2 LLM Generation (LLM Reasoning)

LLM generates:

```
TCP congestion control regulates transmission rate by interpreting
packet loss as a signal of congestion...
```

LLM reasoning internally:

- predicts next tokens
- uses learned knowledge
- generates coherent explanation

3.3 Backend Post Processing

Backend:

- cleans output
- validates length
- stores response

Database:

response_id	task_id
r_201	t1

Same for all tasks.

3.4 Draft Assembly

Backend merges outputs:

```
Draft Answer = r1 + r2 + r3 + r4
```

Workflow state:

```
PLANNED → GENERATED
```

3.5 Next Step Triggered

Claim extraction agent queued.

✅ STEP 3 — GENERATION PHASE

Generator agent executes.

3.1 Prompt Construction (Backend)

For task 1:

```
SYSTEM:

Provide factual technical explanation.


TASK:

Explain TCP congestion control.


CONSTRAINTS:

Avoid speculation. Suitable for verification.
```

3.2 LLM Generation (LLM Reasoning)

LLM generates:

```
TCP congestion control regulates transmission rate by interpreting
packet loss as a signal of congestion...
```

LLM reasoning internally:

- predicts next tokens
- uses learned knowledge
- generates coherent explanation

3.3 Backend Post Processing

Backend:

- cleans output
- validates length
- stores response

Database:

response_id	task_id
r_201	t1

Same for all tasks.

3.4 Draft Assembly

Backend merges outputs:

```
Draft Answer = r1 + r2 + r3 + r4
```

Workflow state:

```
PLANNED → GENERATED
```

3.5 Next Step Triggered

Claim extraction agent queued.

✅ STEP 4 — CLAIM ANALYSIS PHASE

Claim extraction agent starts.

4.1 Input

Draft generated answer.

4.2 Claim Extraction (LLM Reasoning)

LLM receives:

```
Extract factual claims from text.
```

Output:

```
C1: TCP interprets packet loss as congestion.

C2: Wireless networks experience packet loss due to interference.

C3: QUIC operates over UDP.

C4: QUIC reduces connection establishment latency.
```

4.3 Backend Processing

Claims stored individually:

claim_id	text
c1	TCP interprets packet loss as congestion
c2	Wireless packet loss due to interference

Workflow state:

```
GENERATED → CLAIMS_EXTRACTED
```

4.4 Next Step Triggered

Retriever agent queued.

✅ STEP 5 — RETRIEVAL PHASE (DETAILED TECHNICAL EXPLANATION)

✅ 5.0 Purpose of Retrieval Phase

Input to this phase:

```
Claim:

"TCP interprets packet loss as congestion."
```

Goal:

```
Find evidence text that may support or contradict this claim.
```

Important:

Retrieval does NOT verify truth.

It only answers:

> “Which documents are relevant to this claim?”

Verification happens in Step 6.

✅ 5.1 Inputs to Retrieval Phase

Retriever receives:

A. Claim Text

```
TCP interprets packet loss as congestion
```

B. Claim Metadata

(optional)

```
entities = ["TCP", "packet loss", "congestion"]

domain = networking
```

C. Knowledge Sources

Stored earlier:

- Wikipedia chunks
- RFC documents
- Technical docs
- Research papers

✅ 5.2 Internal Retrieval Pipeline

Retrieval is NOT one search.

It is **multi-stage retrieval**.

Pipeline:

```
Claim

    ↓

Embedding Generation

    ↓

Vector Search (Semantic)

    ↓
```

```
Keyword Search (BM25)
    ↓
Result Fusion
    ↓
Top-K Evidence Selection
```

✅ 5.3 Stage 1 — Embedding Generation

Model Used

Embedding model converts text → vector.

Common models:

- `sentence-transformers/all-mpnet-base-v2`
- `BAAI/bge-base-en`
- OpenAI embeddings (optional)

Recommended for project:

`sentence-transformers/all-mpnet-base-v2`

What Happens Internally

Claim:

```
TCP interprets packet loss as congestion
```

Converted to vector:

```
[0.23, -0.81, 0.44, ...]   (768 dimensions)
```

This vector represents meaning.

Why Embeddings?

Because exact words may differ:

Claim:

```
TCP treats packet loss as congestion
```

Evidence:

```
TCP assumes loss indicates congestion
```

Keyword match fails, semantic match succeeds.

✅ 5.4 Stage 2 — Vector Search (Semantic Retrieval)

Tool Used

Vector database:

- FAISS (recommended)
- Weaviate

- Pinecone

Operation

Backend runs:

```
vector_db.search(claim_vector, top_k=10)
```

Internally FAISS:

- uses Approximate Nearest Neighbor search
- finds closest vectors in embedding space

Output Example

```
Chunk A (RFC 5681) similarity = 0.89

Chunk B (Wikipedia) similarity = 0.86

Chunk C (Blog) similarity = 0.62
```

Low similarity chunks discarded.

✅ 5.5 Stage 3 — Keyword Retrieval (BM25)

Semantic search alone is risky.

Example:

Claim:

```
TCP window reduction
```

Semantic search may retrieve congestion topics but miss exact RFC section.

So second retrieval is added.

Model Used

BM25 algorithm (Elasticsearch / Whoosh)

BM25 is NOT AI — it is statistical ranking.

Operation

Backend runs:

```
bm25.search(claim_text)
```

Ranks documents by:

- keyword frequency
- term importance
- document length normalization

Output Example

```
RFC 5681 (high keyword match)

TCP congestion Wikipedia page

Networking tutorial
```

✅ 5.6 Stage 4 — Hybrid Retrieval (Fusion)

Now system has:

```
Vector Results

+

BM25 Results
```

Backend merges them.

Example scoring:

```
final_score =

0.6 * embedding_similarity +

0.4 * bm25_score_normalized
```

This prevents:

- semantic drift
- keyword-only errors

This is called **Hybrid Retrieval**.

Used in production RAG systems.

✅ 5.7 Stage 5 — Evidence Selection

From merged list:

Select:

```
Top 3–5 evidence chunks
```

Example:

chunk	source	score
RFC 5681 section	official	0.91
Wikipedia TCP	encyclopedia	0.88
networking book	textbook	0.85

Stored in database.

✅ 5.8 Backend Execution Flow (Realistic)

Actual flow:

```
for claim in claims:

    claim_vector = embed(claim)
```

```
semantic_results = faiss.search(claim_vector)

keyword_results = bm25.search(claim)

merged_results = combine(semantic_results, keyword_results)

top_evidence = select_top_k(merged_results)

store_evidence(top_evidence)
```

✅ 5.9 Why Two Retrieval Methods Are Needed

Method	Strength	Weakness
Embeddings	understands meaning	may retrieve related but wrong text
BM25	exact keyword match	misses paraphrases

Hybrid = reliable.

✅ 5.10 Example Full Retrieval Dry Run

Claim:

```
QUIC reduces connection establishment latency.
```

Vector search finds:

```
"QUIC uses 0-RTT handshake..."
```

BM25 finds:

```
"QUIC handshake latency improvement"
```

Merged result → correct evidence.

✅ 5.11 Output of Retrieval Phase

Retriever outputs:

```
{
  "claim_id": "c3",
  "evidence": [
    {
      "text": "...QUIC reduces handshake latency...",
      "source": "RFC 9000",
      "similarity": 0.88
    }
```

```
  ]
}
```

This is passed to Verification Phase.

✅ IMPORTANT UNDERSTANDING

Retrieval answers:

```
Where might the answer exist?
```

Verification answers:

```
Does this evidence actually support the claim?
```

Two completely different jobs.

✅ STEP 6 — VERIFICATION PHASE

Core reliability step.

6.1 Relevance Check

Compute similarity:

```
sim(claim, evidence) = 0.88
```

Evidence accepted.

6.2 Entailment Check (NLI Model)

Input:

```
Premise: TCP reduces rate on packet loss.

Hypothesis: TCP interprets packet loss as congestion.
```

Model output:

```
Entailment = 0.91

Neutral = 0.06

Contradiction = 0.03
```

6.3 Credibility Score

Source: RFC → 0.95

6.4 Verification Score Calculation

```
score =

0.5*0.91 + 0.3*0.88 + 0.2*0.95

= 0.909
```

Result:

```
Status = VERIFIED
```

Stored in DB.

Workflow state:

```
EVIDENCE_RETRIEVED → VERIFIED
```

6.5 Next Step Triggered

Critic agent queued.

✅ STEP 7 — CRITIQUE PHASE

Critic receives:

- draft answer
- verification table

7.1 Backend Rule Check

Example:

```
Claim C2 = UNCERTAIN
```

Refinement required.

7.2 LLM Critic Reasoning

LLM receives:

```
Identify weaknesses based on verification results.
```

Output:

```
Explanation lacks distinction between congestion loss and wireless
loss.
```

Critique stored.

✅ STEP 8 — REFINEMENT PHASE

Refinement agent executes.

8.1 Prompt Construction

```
Rewrite answer using verified claims.

Add explanation for wireless loss differences.

Include citations from evidence.
```

8.2 LLM Output

Improved answer:

```
TCP assumes packet loss indicates congestion [RFC 5681].

Wireless networks often experience loss due to interference...

QUIC reduces handshake latency...
```

8.3 Evidence Alignment

Backend checks:

- every sentence linked to verified claim
- citations inserted from evidence DB

Workflow state:

```
VERIFIED → REFINED
```

✅ STEP 9 — COMPLETION PHASE

Backend prepares final output.

9.1 Final Response Object

```
{
  "answer": "...final refined explanation...",
  "claims": [
    {"text":"TCP interprets packet loss as congestion",
```

```
      "status":"verified",
      "source":"RFC 5681"}
  ],
  "confidence_score": 0.89
}
```

9.2 Response Returned to User

Frontend displays:

- final answer
- citations
- verified indicators
- confidence score

Workflow state:

```
COMPLETED
```

✅ FINAL PIPELINE SUMMARY

```
User Query
   ↓
Workflow Initialization
```

```
    ↓
Planner (task creation)
    ↓
Generator (draft creation)
    ↓
Claim Extraction (atomic facts)
    ↓
Retriever (evidence search)
    ↓
Verifier (logical validation)
    ↓
Critic (weakness detection)
    ↓
Refiner (improved answer)
    ↓
Evidence Alignment
    ↓
Final Verified Output
```